Path Planning Assignment
E9: Cuckoo Search & H9: Bidirectional A*

Kalpana Bhaskar – Roll No. 24011101049 – Environment: Map 2

1 Problem Setup

- Grid: 20×20, cells (x, y) with $0 \le x, y \le 19$.
- Start $S = (1, 1)$, Goal $G = (18, 18)$.
- Obstacles: the black blocks in Map 2.
- Movement: 8-connected. Orthogonal moves cost 1, diagonal moves cost $\sqrt{2}$.

2 H9: Bidirectional A*

2.1 Idea

Two A* searches run at once. The forward search starts at S with $h_F(n) = \text{octile}(n, G)$, and the backward search starts at G with $h_B(n) = \text{octile}(n, S)$. Each node is scored by $f(n) = g(n) + h(n)$, where g is the cost so far.

2.2 Steps

1. Initialise $g_F(S) = 0$, $g_B(G) = 0$, open lists $O_F = \{S\}$, $O_B = \{G\}$, and $\mu = \infty$.
2. While both open lists are non-empty and $\min(f^F_{\text{top}}, f^B_{\text{top}}) < \mu$:
   (a) Pick the side with the smaller open list and pop its node n with the lowest f.
   (b) Mark n as closed.
   (c) For each legal neighbour m (no obstacle, no corner cutting):
   - $g_{\text{new}} = g(n) + c(n, m)$. If $g_{\text{new}} < g(m)$, update $g(m)$, set $\text{parent}(m) = n$, and push m with $f = g(m) + h(m)$.
   - If m has already been reached by the other search, the candidate cost is $g_{\text{side}}(m) + g_{\text{other}}(m)$. If this is less than μ, set μ and $\text{meet} = m$.
3. Path = (backtrack from meet to S via forward parents) + (backtrack from meet to G via backward parents).

2.3 Worked First Expansion (Forward Search)

The start node $S = (1, 1)$ is expanded first. Its 8 neighbours have these values, with $h(n) = \text{octile}(n, G)$:

Neighbour m	$g(m)$	$h(m)$	$f(m)$
$(2, 2)$	1.414	22.627	**24.041**
$(2, 1)$	1.000	23.627	24.627
$(1, 2)$	1.000	23.627	24.627
$(1, 0)$	1.000	25.041	26.041
$(0, 1)$	1.000	25.041	26.041
$(2, 0)$	1.414	24.627	26.041
$(0, 2)$	1.414	24.627	26.041
$(0, 0)$	1.414	25.454	26.868

The node with the lowest f is $(2, 2)$ with $f = 24.041$, so it is the next node to expand. Its parent is S, and it was reached diagonally with no corner cutting because $(2, 1)$ and $(1, 2)$ are free.

3 E9: Cuckoo Search

3.1 Encoding

Each nest holds $N = 6$ intermediate waypoints, giving a vector $x \in [0, 19]^{12}$. The full path is $S \to w_1 \to \cdots \to w_6 \to G$.

3.2 Fitness

$$f(x) = L(x) + \lambda\, C(x),$$

where $L(x)$ is the polyline length, $C(x)$ is the number of colliding segments, and $\lambda = 50$. A segment collides if any sampled point on it falls inside an obstacle.

3.3 Algorithm Steps

1. Initialise $n = 25$ random nests and evaluate their fitness. Set x_{best} to the best nest.
2. For each nest x_i, generate a new solution with a Lévy flight:

   $$x_i' = x_i + \alpha \cdot \text{Lévy}(\beta) \odot (x_i - x_{\text{best}}), \qquad \alpha = 0.1,\ \beta = 1.5.$$

   Clip x_i' to the grid.
3. Pick a random nest j. If $f(x_i') < f(x_j)$, replace x_j with x_i'.
4. Abandon each nest with probability $p_a = 0.25$ and replace it with a new random nest.
5. Update x_{best} if a better nest exists.
6. Repeat steps 2–5 for 300 iterations.

Lévy steps use Mantegna's method:

$$\sigma_u = \left[\frac{\Gamma(1+\beta)\sin(\pi\beta/2)}{\Gamma\left(\frac{1+\beta}{2}\right)\beta\, 2^{(\beta-1)/2}} \right]^{1/\beta}, \qquad \text{step} = \frac{u}{|v|^{1/\beta}}, \quad u \sim \mathcal{N}(0, \sigma_u^2),\ v \sim \mathcal{N}(0, 1).$$

4 Evaluation Metrics

- **Path length:** sum of Euclidean segment lengths.
- **Computation time:** wall-clock time (seconds) per run.
- **Success rate:** percentage of runs that end with a collision-free path.
- **Number of turns:** interior vertices where the heading changes by more than 5°.
- **Collision-free navigation:** checked by sampling every segment.

Algorithm	Path length	Time (s)	Success rate (%)	Turns
E9: Cuckoo Search (mean, 10 runs)	54.798	6.109	30.0	5.7
H9: Bidirectional A*	27.556	0.00218	100	7

Table 1: Map 2 results

5 Results

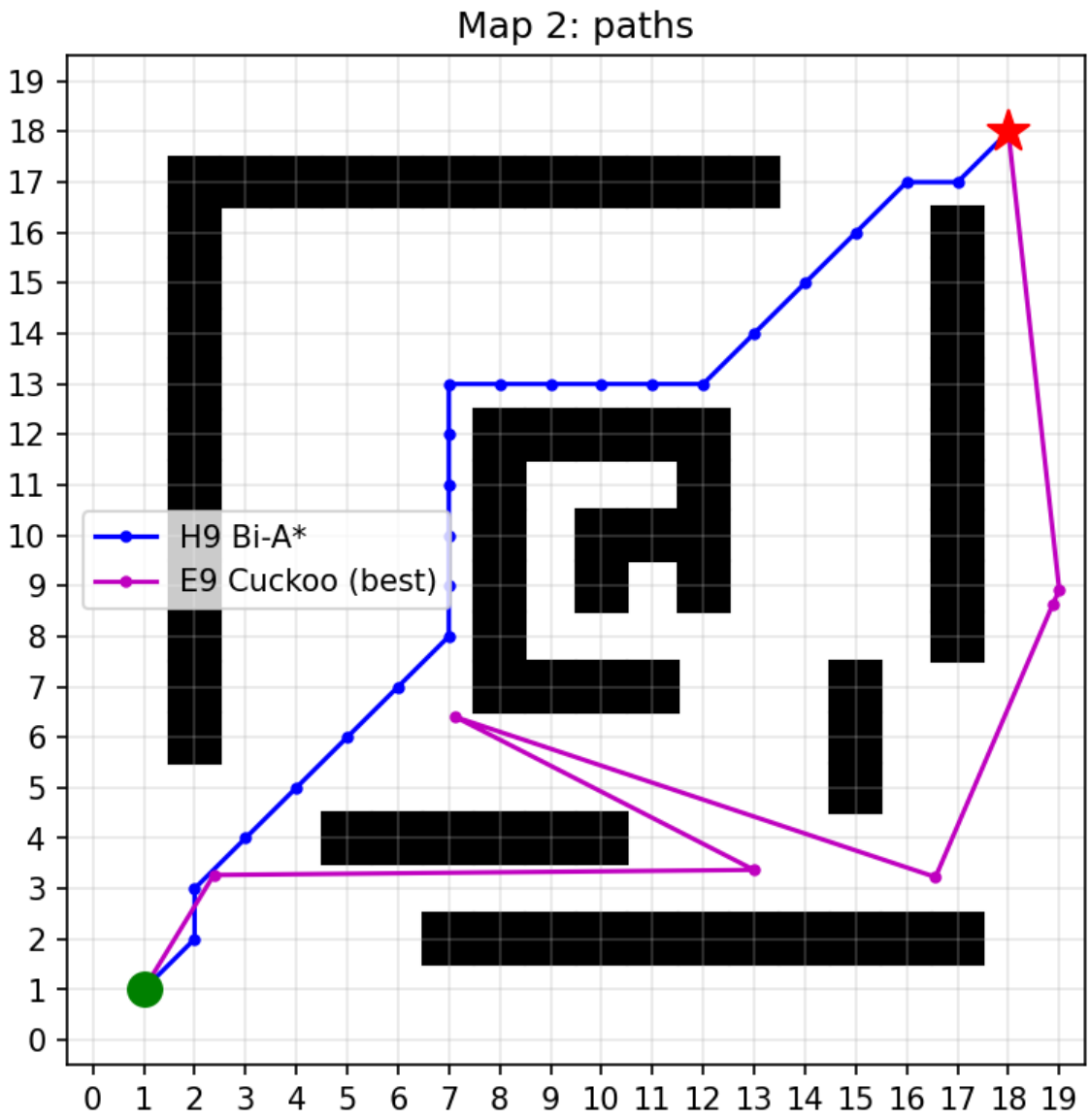


Figure 1: A descriptive caption for your figure explaining what it shows.

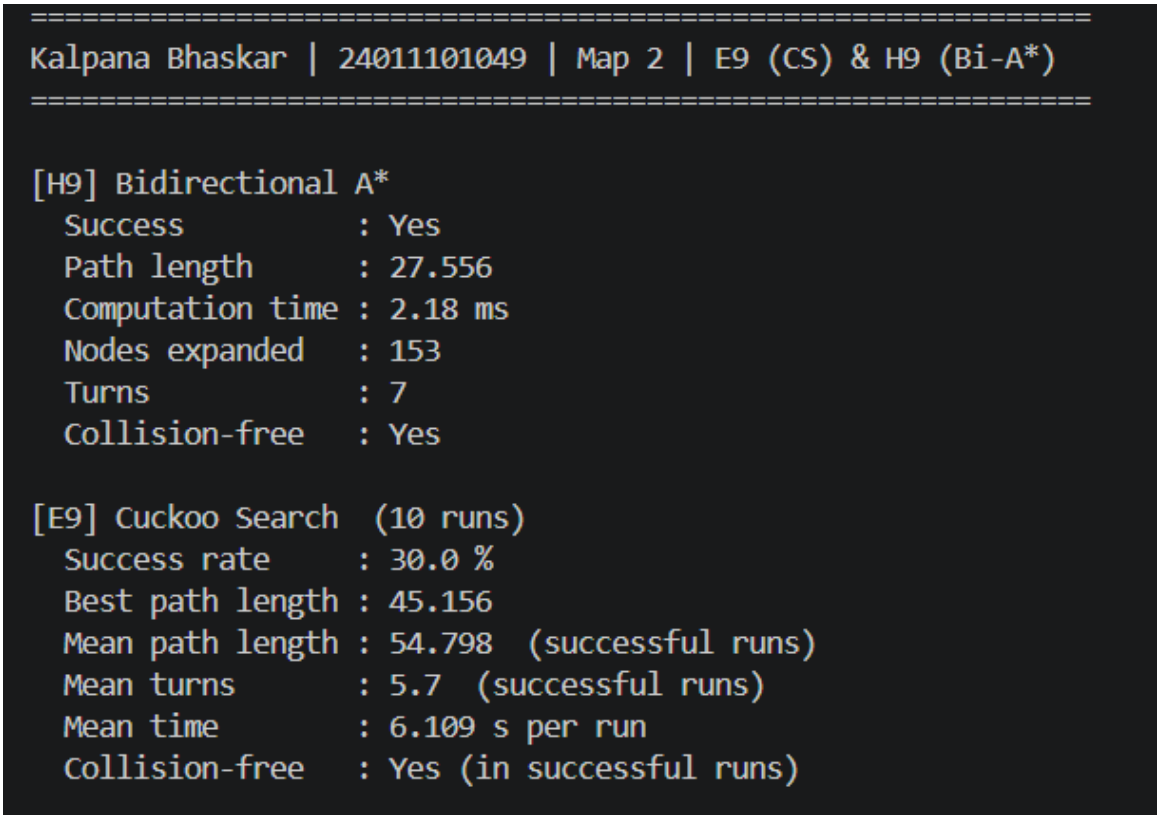


Figure 2: A descriptive caption for your figure explaining what it shows.

6 Discussion

7 Discussion

- **Optimality Comparison:** Bidirectional A* (H_9) guarantees the shortest discrete grid path (27.556 units) because it explores the 8-connected grid exhaustively until a meeting point is found. In contrast, Cuckoo Search (E_9) produces a smooth, continuous path defined by intermediate waypoints; however, due to stochastic metaheuristic optimization and the risk of getting trapped in local optima over 300 iterations, it yields a longer mean path length (54.798 units) and lower consistency.

- **Time and Success Rate Analysis:** Bidirectional A* is computationally lightweight, executing in just 2.18 ms with a 100% success rate. Conversely, Cuckoo Search requires significantly higher overhead, averaging 6.109 s per run due to repeated population updates, Lévi flight vector computations, and segment collision checks across 25 nests. The stochastic nature of Cuckoo Search causes performance to vary widely between runs; because initialization and random walk steps differ each time, the algorithm occasionally fails to navigate complex obstacle traps, resulting in a low success rate of 30.0%.

- **Impact of Penalty Weight λ and Waypoints:** The penalty parameter λ ($= 50$) heavily dictates path feasibility by penalizing obstacle collisions in the fitness function $f(x) = L(x) + \lambda C(x)$. If λ is set too low, the algorithm may favor shorter paths that illegally cut through obstacles. If λ is excessively high, the search becomes overly constrained, freezing progress. Similarly, the number of waypoints ($N = 6$) defines the trajectory's resolution: too few waypoints prevent the path from flexibly maneuvering around tight obstacle corners, while too many waypoints drastically increase the search space dimensionality, making it harder for the metaheuristic to converge on a collision-free solution within the iteration limit.